

Developer Series 2

UI/UX Production, Figma MCP & Motion Pipeline

Purpose. Define how Vossie's confirmed screen architecture becomes a consistent, editable Figma system and then a responsive, animated working prototype without making paid tooling a core dependency.

Scope. Decision 01: Figma + AI design-to-development pipeline. Decision 02: application motion and microinteractions using Motion.dev. Decision 03: optional generative motion/video using Motion.so. Zero-cost prototype operation is a hard constraint.

177

screen designs

One design language. Reusable components. Adaptive phone/tablet/desktop behaviour. Motion is systematic, not independently invented per screen.

Pipeline

Vossie specifications + existing prototype -> ChatGPT coordination -> Figma design system -> 177 native screen designs -> responsive states -> Motion system -> Cursor implementation -> VS Code QA/security/testing -> shared PWA.

September 2026 - Version 1

Decision 01 - Figma + AI design-to-development pipeline

Figma becomes the UI/UX source of truth. The objective is not to generate 177 disconnected mockups. The system is built from tokens, primitives, reusable components and role-specific patterns, then assembled into the confirmed screen families.

Layer	What is produced	Rule
Foundation	Colour, typography, spacing, radius, elevation, icon and grid tokens	Token first; no arbitrary one-off styling
Primitives	Buttons, inputs, selectors, chips, tabs, loaders, dialogs, sheets	Accessible states included
Marketplace	Listing/request cards, seller modules, cart/checkout, order states	Shared across Buyer/Seller where possible
Logistics	Courier job cards, map overlays, QR/custody, transfer timeline	State-driven; map and custody truth stay separate
Operations	Tables, filters, moderation, finance, hub and incubation modules	Desktop-first but adaptive
Screen assembly	Access, Shared, Buyer, Seller, Courier, Hub, Transfers, Incubation, Operations, Extensions	Reuse components before creating new ones

Figma MCP operating loop

1. ChatGPT maintains screen architecture, interaction rules, naming and acceptance criteria. 2. Figma MCP reads the existing library and, where the account permits write access, creates or updates native frames, components, variables and Auto Layout. 3. Designs are reviewed in Figma. 4. Cursor receives Figma context and implements the corresponding React components. 5. VS Code is the independent verification environment. 6. Differences return to Figma/specification or code as an explicit correction rather than silent drift.

Responsive rule

The same product behaviour is expressed differently by available space: phone uses mobile-first navigation and sheets; tablet uses split/contextual views; desktop uses sidebars, multi-column layouts, larger maps and operational tables. Responsive variants share semantics, tokens and component contracts.

Account/cost constraint

Figma's remote MCP can provide design context broadly, but write-to-canvas availability depends on Figma seat/plan conditions and is currently beta. Therefore automated Figma writing is an accelerator, not a requirement for Vossie to function. If write access is unavailable, the same design specification and components remain usable manually and by code-generation workflows.

Decision 02 - Vossie motion & microinteraction system

Motion must communicate hierarchy, state and causality without turning Vossie into a visually noisy interface. The core application animation layer is Motion.dev/CSS. Motion.dev's free AI Kit skill and documentation search can be connected to Cursor; premium Motion+ capabilities are optional and must not be required.

Pattern	Vossie examples	Default intent
Field response	Focus, validation, password reveal, verification state	Immediate feedback
Button/process	Press, loading, success, disabled, retry	Make system state obvious
Navigation	Tab indicator, bottom-nav change, sidebar selection	Preserve orientation
Cards	Listing expansion, request offer, courier job card	Show relationship between states
Overlays	Sheet, dialog, filter, checkout step	Maintain spatial context
Lists	Add/remove/reorder, notifications, order updates	Explain data changes
Progress	Checkout, upload, transfer/custody, onboarding	Show progression, not decoration
Maps/logistics	Courier marker, route status, pickup/delivery milestone	Subtle operational feedback
Success	Order placed, QR accepted, delivery complete	Short confirmation

Reusable motion tokens

Enter / Exit / Tab Shift / Sheet Up / Card Lift / Expand / Collapse / Progress / Map Pulse / Success / Stagger / Counter / Reduced Motion. Each token gets approved duration/easing/physics and is reused across roles. Screen designers choose from the vocabulary instead of inventing new animation behaviour.

Implementation pipeline

Figma state/interaction -> motion specification -> Motion.dev/CSS implementation in Cursor -> browser preview -> VS Code/Playwright verification -> accessibility/performance check -> reusable component approved. Motion.dev's free agent skill and current-documentation MCP search support the implementation. Premium examples, MotionScore and advanced editor features remain optional.

Accessibility and performance gates

Respect prefers-reduced-motion. Never make animation the only carrier of meaning. Prefer transform/opacity for routine movement, avoid unnecessary layout-triggering animation, keep input feedback fast, and do not block navigation or transaction completion on decorative motion. Repeated animations must remain calm enough for sustained marketplace use.

Decision 03 - Motion.so generative motion layer

Motion.so (Mosaic Motion) is separate from Motion.dev. It is an AI motion-video system with an MCP/API workflow. It can generate motion videos from prompts and can accept design guidance and reference assets. This is useful for showpiece assets, but its API/MCP jobs consume credits, so it is not a core prototype dependency.

Appropriate uses

Vossie logo/identity animation; short onboarding or feature explainers; presentation/showcase sequences; launch visuals; optional animated assets that can be exported and embedded without requiring Motion.so at runtime.

Do not use it for

Login-field feedback, navigation, loading indicators, cards, checkout state, courier interaction, operational dashboards or any other functional UI behaviour. Those remain native application interactions using CSS/Motion.dev.

Zero-budget rule

R0 core dependency budget: no paid service is required for the prototype to operate. Motion.so output may be used only when free/sponsored credits are available or when an asset has already been generated and exported. Loss of Motion.so access must not break a Vossie flow.

Tool responsibility matrix

Tool	Primary responsibility	Required?
ChatGPT	Architecture, screen contracts, interaction/motion specifications, coordination	Core coordination
Figma	Design system, editable UI/UX, responsive states, prototype review	Core design source
Figma MCP	Design context; native canvas creation/update where account access permits	Accelerator
Motion.dev	Runtime microinteractions and React/CSS animation patterns	Core motion layer
Motion.dev AI Kit	Free skill/docs context for coding agents; premium features optional	Free core / optional premium
Motion.so	Generated motion-video/showcase assets	Optional only
Cursor	Implementation agents for React/UI/motion	Core implementation
VS Code	Review, testing, security, accessibility, performance, Git gates	Core verification

Locked Developer Series 2 pipeline

Specifications -> Figma design system -> screen families -> responsive states -> motion specification -> Figma context/MCP -> Cursor + Motion.dev implementation -> VS Code QA -> working PWA. Motion.so sits beside this pipeline as an optional exported-media generator, never inside the critical runtime path.

This document extends the existing Developer Series architecture; it does not replace Developer Series 1. Series 1 defines the application/agent/stack foundation. Series 2 defines how the UI/UX and motion system are produced and handed into that

foundation.